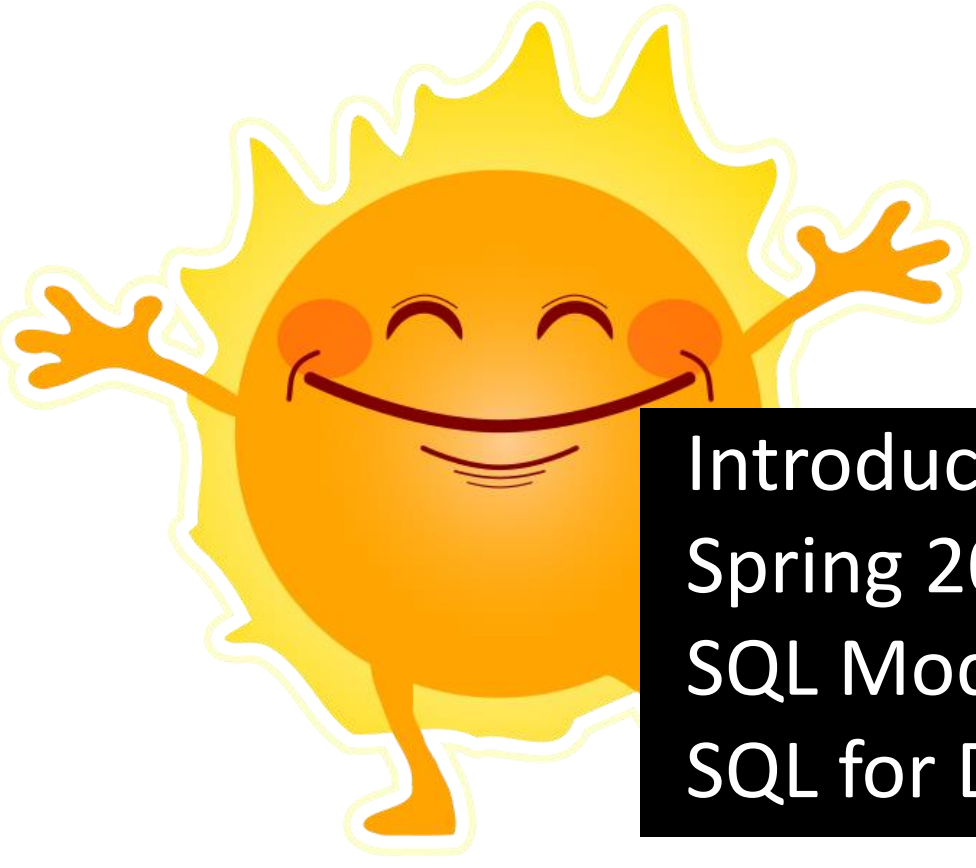


Introduction to Database Systems
Spring 2025, Lecture 5:
SQL Modifications and
SQL for Data Analytics

Source: Sunshine - Openclipart

Phones out!

How was last week's lecture on SQL?

www.menti.com Code: 7256 1172

Repeat: Most Expensive Products

Product (<u>ProductName</u>, Price)

ProductName	Price
Tea	100
Small	85
Large	100
Fancy	NULL

```
-- wrong: using limit and order by
-- (misses Tea)
select *
from product
order by price desc, productname
limit 1;
```

A-Z productname	123 price
Large	100

```
-- wrong: aggregate function without
-- group by (does not compile)
select productname, max(price)
from product;
```

SQL Error: java.sql.SQLException: Binder Error: column "productname" must appear in the GROUP BY clause or must be part of an aggregate function.

```
-- wrong: outputs all groups and
-- max price per group
select productname, max(price)
from product
group by productname;
```

A-Z productname	123 max(price)
Large	100
Fancy	[NULL]
Tea	100
Small	85

Repeat: Most Expensive Products

Product (ProductName, Price)

ProductName	Price
Tea	100
Small	85
Large	100
Fancy	NULL

```
-- correct: most expensive products
select *
from product
where price = (select max(price) from product)
order by productname;
```

A-Z productname	123 price
Large	100
Tea	100

First: find max price

Second: select all products with this max price

Intended Learning Outcomes

1. Suggest a database design in the ER model and convert to a relational database schema in a suitable normal form.
2. Write SQL queries, involving multiple relations, compound conditions, grouping, aggregation, and subqueries.
3. Use relational DBMSs from a conventional programming language in a secure manner.
4. Analyze/predict/improve query processing efficiency of the designed database using indices.
5. Reflect upon the evolution of the hardware and storage hierarchy and its impact on data management system design.
6. Discuss the pros and cons of different classes of data systems for modern analytics and data science applications.

Agenda

SQL Modifications (DMLs and DDLs)

- Insert, update, delete
- Add / remove columns

SQL for Data Analytics

- Views
- WITH clause
- Window queries
- User-defined functions
- LLMs

Coffeehouse Database (continued from Lecture 4)

Shift (Day, Cashier)

Day	Cashier
Feb 11	Mads
Feb 12	Anna
Feb 18	Anna
Feb 19	Anna

Product (ProductName, Price)

ProductName	Price
Tea	100
Small	85
Large	100
Fancy	NULL

User (UserName, Email, Password)

UserName	Email	Password
Anna	anna	***
Martin	mhent	***
Omar	omsh	***

Purchase (PurchaseTime, ProductName, UserName)

PurchaseTime	ProductName	UserName
Feb 11, 9:55	Tea	Martin
Feb 12, 10:03	Small	Martin
Feb 12, 10:05	Small	Omar
Feb 12, 10:06	Large	Omar
Feb 19, 9:00	Small	Martin

Coffeehouse Database, continued from Lecture 4

Attribute types, same as in Lecture 4

```
create table shift(day date, cashier string);
create table user(username string, email string, password string);
create table product(productname string, price int);
create table purchase(purchasetime timestamp, productname string,
                      username string);
```

Again, SQL queries of this lecture can be downloaded from LearnIT

- lecture5.sql

Agenda

SQL Modifications (DMLs and DDLs)

- Insert, update, delete
- Add / remove columns

SQL for Data Analytics

- Views
- WITH clause
- Window queries
- User-defined functions
- LLMs

Insert

User (<u>UserName</u>, Email, Password)

UserName	Email	Password
Anna	anna	***
Martin	mhent	***
Omar	omsh	***

The **insert** statement adds data to a table.

```
-- insert full rows
insert into user values
  ('Mads', 'mads@itu.dk', '***'),
  ('Christina', 'chris@itu.dk', '***');

select * from user;
```

A-Z username	A-Z email	A-Z password
Anna	anna	***
Martin	mhent	***
Omar	omsh	***
Mads	mads@itu.dk	***
Christina	chris@itu.dk	***

Insert Partial Data

```
-- insert only specific columns
insert into user(username) values ('Donna');
```

A-Z username	A-Z email	A-Z password
Anna	anna	***
Martin	mhent	***
Omar	omsh	***
Mads	mads@itu.dk	***
Christina	chris@itu.dk	***
Donna	[NULL]	[NULL]

Specify one or multiple attribute names

Remaining attributes are set to NULL

Insert Combined with Select

```
-- create a table with unique email addresses
create table email(email string primary key);

-- insert combined with subquery
insert into email
select email from user where email is not null;

select *
from email;
```

A-Z email
anna
mhent
omsh
mads@itu.dk
chris@itu.dk

Insert and select in one statement

Reminder: Primary key ensures uniqueness (no duplicates)

Insert and Conflicts

```
-- insert a duplicate email address (error)
insert into email values ('mads@itu.dk');
```

```
SQL Error: Constraint Error: Duplicate key
"email: mads@itu.dk" violates primary key
constraint. If this is an unexpected constraint
violation please double check with the known
index limitations section in our documentation
(https://duckdb.org/docs/sql/indexes).
```

```
-- specify what to do, e.g. ignore or update
insert into email values ('mads@itu.dk')
on conflict do nothing;
```

Name	Value
Updated Rows	0

No change to the table, but also no error, which is good because the SQL query returns normally, and your application won't run into an exception

Update

Modifications to data can be made using **update**.

```
-- update user Donna to add email and pw
update user
set email = 'donna@itu.dk', password = '***'
where username = 'Donna';

select * from user;
```

A-Z username	A-Z email	A-Z password
Anna	anna	***
Martin	mhent	***
Omar	omsh	***
Mads	mads@itu.dk	***
Christina	chris@itu.dk	***
Donna	donna@itu.dk	***

Attention: if you forget to use the where clause, you will update all rows in the table.

Live Exercise 1

Update the user table to correct incomplete email addresses by adding the "@itu.dk" suffix. Use concat(string, …) to concatenate strings.

username	email	password
Anna	anna	***
Martin	mhent	***
Omar	omsh	***
Mads	mads@itu.dk	***
Christina	chris@itu.dk	***
Donna	donna@itu.dk	***

username	email	password
Anna	anna@itu.dk	***
Martin	mhent@itu.dk	***
Omar	omsh@itu.dk	***
Mads	mads@itu.dk	***
Christina	chris@itu.dk	***
Donna	donna@itu.dk	***

```
update user
set email = concat(email, '@itu.dk')
where email not like '%@itu.dk';
```

Delete

Data can be removed using the **delete** statement.

Product (ProductName, Price)

ProductName	Price
Tea	100
Small	85
Large	100
Fancy	NULL

```
-- delete the Fancy product
delete from product
where productname = 'Fancy';

select * from product;
```

productname	price
Tea	100
Small	85
Large	100

```
-- delete everything from product
delete from product;

select count(*) from product;
```

count_star()
0

= empty!

Adding and Removing Columns

Adding rows → **insert**
Deleting rows → **delete**

How about adding and deleting columns?

Needs a DDL:

```
alter table <table_name> add column <column_name> <type>;
alter table <table_name> drop column <column_name>;
```

Example

Let's hash the cleartext passwords, including salt[1], and remove the cleartext passwords from the user table.

```
-- add password_hash and salt to user table
alter table user add column password_hash string;
alter table user add column salt double;

-- generate random salt values, and hash passwords
update user set salt = random();
update user set password_hash = sha256(concat(password, salt));

-- drop the cleartext passwords
alter table user drop column password;
```

[1]Salt is a random value added to passwords before hashing to ensure that identical passwords produce different hashes.

Example ctd.

User table after these changes:

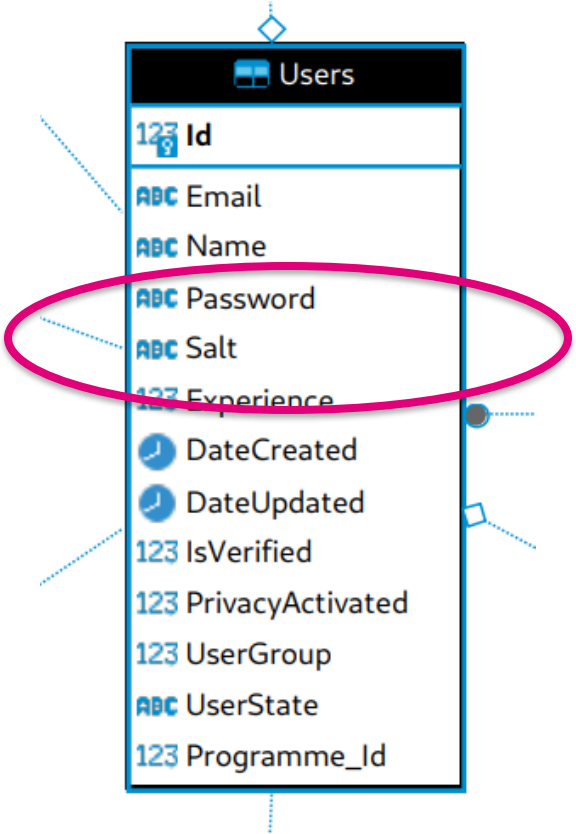
A-Z username	A-Z email	A-Z password_hash	123 salt
Anna	anna@itu.dk	9be187dc94e7cf51eb15789110f64ae2557c09ecf292e48ec9da63be995049df	0.8401533393
Martin	mhent@itu.dk	c319bbc603c6b26c9a64eec7d912d64f8dabd2e40d363ea6af453a225e93a91a	0.7724605463
Omar	omsh@itu.dk	048a51145a35bd25445061427045cd6270e1afa3aee9c56302e503fd33b3419f	0.8049573221
Mads	mads@itu.dk	37e1a7893902e174a49b3828ccd4c4c3589e961df239f7d755ea303a48e11e4d	0.5786613324
Christina	chris@itu.dk	eed50a45d62ac826ed7daa73bbf969b2073c454accda9fc3979f2631e2436af7	0.2622541897
Donna	donna@itu.dk	89d85823db62152137ca72368ac1ad08246620a7e395b83b7c62420f23ac7fc3	0.284384985

Better than before:

- Same cleartext password “***” now stored differently for every user. An attacker doesn’t know that it is the same password.
- Hard to revert the hash value. If database gets leaked, hard to compute cleartext passwords.
- How to test correct password entry by a user that wants to log in? Hash the user input and salt, compare to hash in the password_hash column.

Example ctd.

Remember the database schema of the Analog app from Exercise 3? They also use hashed passwords with salt:



Example ctd.

Disclaimer:

- In the industry, SHA-256 is not good enough.
- Use at least 16 random bytes from a cryptographically secure random number generator.
- Use a hash function that is stronger than SHA-256, e.g. bcrypt or Argon2.
- bcrypt and Argon2 are not natively supported in SQL, we either need a user-defined function (see later slides) or compute the hash value outside of SQL (next lecture).

Live Exercise 2

Donna changes her password from *** to fL7mK2jZ9c.
Update the password_hash values in the user table.
Compute the hash using: `sha256(concat('fL7mK2jZ9c', salt))`

```
update user set password_hash = sha256(concat('fL7mK2jZ9c', salt))
where username = 'Donna';
```

Test that the user input fL7mK2jZ9c is correct.

```
select password_hash = sha256(concat('fL7mK2jZ9c', salt))
from user
where username = 'Donna';
```

Agenda

SQL Modifications (DMLs and DDLs)

- Insert, update, delete
- Add / remove columns

SQL for Data Analytics

- Views
- WITH clause
- Window queries
- User-defined functions
- LLMs

Views

A view is a virtual table based on the result set of a SQL query

- The view is stored in the database
- Queried like a regular table

Benefits:

- Ease of use: hide complex queries from users
- Security: can restrict access to specific rows or columns, using RBAC (Lecture 7)

Create View

View that computes revenue per day

```
create view revenuePerDay as
select purchasetime::date as day, sum(price) as revenue
from purchase join product
on purchase.productname = product.productname
group by day;
```

Query like a regular table

```
select * from revenuePerDay
where revenue >= 100
order by revenue desc;
```

day	123 revenue
2025-02-12 00:00:00.000	270
2025-02-11 00:00:00.000	100

Behind the Scenes: Query Rewrite

The database rewrites the query to insert the view definition.

```
select * from revenuePerDay
where revenue >= 100
order by revenue desc;
```

```
select * from (
  select purchasetime::date as day, sum(price) as revenue
  from purchase join product
  on purchase.productname = product.productname
  group by day
)
where revenue >= 100
order by revenue desc;
```

WITH Clause

The `WITH` clause define a temporary result set (a Common Table Expression – CTE)

- Not stored in the database, only for a single query
- But can be referenced multiple times in the query

Benefits:

- Ease of use: can improve readability of queries
- Crazy stuff: Supports recursion

Example WITH Statement

Same computation (and result) as before

```
with revenuePerDay as (
  select purchasetime::date as day, sum(price) as revenue
  from purchase join product
  on purchase.productname = product.productname
  group by day
)
select * from revenuePerDay
where revenue >= 100
order by revenue desc;
```

day	123 revenue
2025-02-12 00:00:00.000	270
2025-02-11 00:00:00.000	100

Recursive WITH

Recursion allows to query tree-like structures or graphs.

Simple example*:

```
-- recursive with
with recursive counter(num) as (
  select 1
  union all
  select num + 1 from counter
)
select num from counter
limit 10;
```

Base case

Recursive step, references CTE by name (counter)

Result

123 num
1
2
3
4
5
6
7
8
9
10

*Source: Working with CTEs (Common Table Expressions) | Snowflake Documentation

Views vs. Common Table Expressions (CTEs)

	View	CTE
Definition	A virtual table	A temporary result
Creation	`CREATE VIEW` statement	`WITH` clause
Persistence	Stored in the database. Exists until dropped via `DROP VIEW`.	Not stored in the database. Only exists within the scope of the query.
Reusability	Can be reused across multiple queries and sessions.	Can be referenced multiple times in the same query. Cannot be reused across queries or sessions.
Security	Views can be used to restrict access to rows & columns.	No security benefits.
Recursion	No support.	Supports recursion.

Agenda

SQL Modifications (DMLs and DDLs)

- Insert, update, delete
- Add / remove columns

SQL for Data Analytics

- Views
- WITH clause
- Window queries
- User-defined functions
- LLMs

Window Functions

A window function is a function which uses values from one or multiple rows to return a value for each row.*

- Aggregate functions: reduce multiple rows to a single value (think **GROUP BY**)
- Window functions: retain the original rows and provide additional calculated columns based on a window of rows (always have an **OVER** clause)

*Source: Window function (SQL) - Wikipedia

Window Functions: Key Concepts

Window Definition: The `OVER` clause defines the window of rows for the function to operate on. It specifies how the rows are partitioned and ordered.

Partitioning: The `PARTITION BY` clause divides the result set into partitions to which the window function is applied. Each partition is processed separately.

Ordering: The `ORDER BY` clause within the `OVER` clause defines the logical order of the rows within each partition.

Framing: The `ROWS` or `RANGE` clause specifies the frame of rows within the partition that the function considers. This can be used to define moving averages, running totals, etc.

Common Window Functions

Aggregate Functions: Functions like `SUM()`, `AVG()`, `COUNT()`, `MIN()`, `MAX()` can be used as window functions to perform calculations over a window of rows.

Ranking Functions: Functions like `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()` assign a unique rank or position to each row within a partition.

Analytic Functions: Functions like `LAG()`, `LEAD()`, `FIRST_VALUE()`, and `LAST_VALUE()` provide access to other rows in the window relative to the current row.

Example View

View to make the following examples easier

```
-- create a view for the following examples
create view sales as
select pu.*, price
from purchase pu join product pr
on pu.productname = pr.productname;


select * from sales order by purchasetime;
```

purchasetime	productname	username	price
2025-02-11 09:55:00.000	Tea	Martin	100
2025-02-12 10:03:00.000	Small	Martin	85
2025-02-12 10:05:00.000	Small	Omar	85
2025-02-12 10:06:00.000	Large	Omar	100
2025-02-19 09:00:00.000	Small	Martin	85

Example: Aggregate Window Function

Cumulative sum of sales over time

OVER specifies the window function

```
-- cumulative sum of sales over time
select *, sum(price) over (order by purchasetime) as cum_sales
from sales
order by purchasetime;
```

This order by is only for the window function

purchasetime	productname	username	price	cum_sales
2025-02-11 09:55:00.000	Tea	Martin	100	100
2025-02-12 10:03:00.000	Small	Martin	85	185
2025-02-12 10:05:00.000	Small	Omar	85	270
2025-02-12 10:06:00.000	Large	Omar	100	370
2025-02-19 09:00:00.000	Small	Martin	85	455

The second order by is sorting the result (could be a different order than the window function)

Example: Aggregate + Partition By

```sql
-- cumulative sum of sales per user over time
select *,
  sum(price) over (partition by username order by purchasetime)
  as cum_sales
from sales
order by purchasetime;
```

Cumulative sum is per user now

purchasetime	productname	username	price	cum_sales
2025-02-11 09:55:00.000	Tea	Martin	100	100
2025-02-12 10:03:00.000	Small	Martin	85	185
2025-02-12 10:05:00.000	Small	Omar	85	85
2025-02-12 10:06:00.000	Large	Omar	100	185
2025-02-19 09:00:00.000	Small	Martin	85	270

Example: Ranking window function

Rank products by how well they sold

```
-- rank products by how well they sold
with product_sums as (
  select pr.productname, sum(coalesce(pr.price, 0)) as prod_sales
  from product pr left outer join sales s
  on pr.productname = s.productname
  group by pr.productname
)
select rank() over(order by prod_sales desc) as product_rank, *
from product_sums
order by product_rank, productname;
```

123 product_rank	A-Z productname	123 prod_sales
1	Small	255
2	Large	100
2	Tea	100
4	Fancy	0

Large and Tea had the same sales, therefore rank is the same!

Example: Analytic window functions

Compute difference of sales to previous day

```
with revenue_per_day as (
  -- same as the very last query of lecture 4
  select purchasetime::date as day, sum(price) as revenue
  from purchase join product
  on purchase.productname = product.productname
  group by day
)
select day, revenue,
  revenue-lag(revenue) over(order by day) as diff_to_prev_day
from revenue_per_day;
```

day	revenue	diff_to_prev_day
2025-02-11 00:00:00.000	100	[NULL]
2025-02-12 00:00:00.000	270	170
2025-02-19 00:00:00.000	85	-185

Maybe makes more sense for stock prices, etc. – but you get the point.

Row Framing

```sql
create table results(points int);
insert into results values (10), (8), (12), (9), (9), (9), (5), (7);

-- compute the sum of each point and the points on either side of it
select points,
  sum(points) over (rows between 1 preceding and 1 following) as we
from results;
```

points	we
10	18
8	30
12	29
9	30
9	27
9	23
5	21
7	12

points	we
10	18
8	30
12	29
9	30
9	27
9	23
5	21
7	12

points	we
10	18
8	30
12	29
9	30
9	27
9	23
5	21
7	12

points	we
10	18
8	30
12	29
9	30
9	27
9	23
5	21
7	12

Source: [Window Functions – DuckDB]

Better: Always Use ORDER BY in Window Function

Previous example was from DuckDB documentation

- Uses an implicit sort order
- Not guaranteed in other databases
- Always use **ORDER BY** in window function

```
-- requires an id column in the results table
create table results(id int, points int);
insert into results values
  (1, 10), (2, 8), (3, 12), (4, 9),
  (5, 9),  (6, 9), (7, 5),  (8, 7);

select points,
sum(points) over (order by id
  rows between 1 preceding and 1 following) as we
from results order by id;
```

Important!

Same result

123 points	123 we
10	18
8	30
12	29
9	30
9	27
9	23
5	21
7	12

Agenda

SQL Modifications (DMLs and DDLs)

- Insert, update, delete
- Add / remove columns

SQL for Data Analytics

- Views
- WITH clause
- Window queries
- User-defined functions
- LLMs

User-Defined Functions

User-defined functions (UDFs) are custom functions created by users

- Extend functionality of SQL
- Hide complex operations
  → Better readability
  → Code reuse
- Can be written in SQL, JavaScript, Python, Java, …

UDFs do not exist in DuckDB. The following examples were created using Snowflake.*

*However, see Python Function API – DuckDB. Allows to create UDFs from within Python. But there is no `CREATE FUNCTION` syntax in DuckDB.

SQL UDFs for code reusability, modularity, readability

```
-- SQL UDF to calculate discount by given percent
create function discount(price int, discount_percent int)
returns float
as
$$
  price * (100 - discount_percent)::float / 100
$$;

select productname, price, discount(price, 18), discount(price, 25)
from product;
```

PRODUCTNAME	PRICE	DISCOUNT(PRICE, 18)	DISCOUNT(PRICE, 25)
Tea	100	82	75
Small	85	69.7	63.75
Large	100	82	75
Fancy	null	null	null

JavaScript UDFs

UDFs in JavaScript, Python, Java, etc. extend functionality of SQL in addition to the other benefits such as reusability, modularity, readability

```
create function is_anagram(A string, B string)
returns boolean
language javascript
as
$$
  A = A.toLowerCase();
  B = B.toLowerCase();
  var sorted_a = A.split('').sort().join('');
  var sorted_b = B.split('').sort().join('');
  return sorted_a == sorted_b;
$$;
```

The function body is written in JavaScript

JavaScript UDFs ctd.

```
create table names(name string);
insert into names values
  ('Elina'), ('Naile'), ('Aleni'), ('Marcus'), ('Sophie');

select x.name, y.name
from names x, names y
where is_anagram(x.name, y.name)
  and x.name < y.name
order by x.name;
```

Call to the UDF in the join condition.

NAME	NAME
Aleni	Elina
Aleni	Naile
Elina	Naile

Last example: extend SQL with a Python UDF

```
-- Python UDF to hash a password using bcrypt
create or replace function bcrypt_str(input string)
returns string
language python
runtime_version = '3.9'
packages = ('bcrypt')
handler = 'bcrypt_str'
as
$$
import bcrypt
def bcrypt_str(input):
  hashed = bcrypt.hashpw(input.encode('utf-8'), bcrypt.gensalt())
  return hashed.decode('utf-8')
$$;
```

Import any Python libraries (can be multiple)

Specifies which function to call in the given Python code

Include salt

Python UDF ctd.

User (<u>UserName</u>, Email, Password)

UserName	Email	Password
Anna	anna	***
Martin	mhent	***
Omar	omsh	***

```
-- execute Python UDF on original user table
select *, bcrypt_str(password) from user;
```

USERNAME	EMAIL	PASSWORD	BCRYPT_STR(PASSWORD)
Anna	anna	***	$2b$12$7Uuapstjt/QvUBHxzeoc7enKUncDReq8xgpeGiwHYIjuPS.VGvfom
Martin	mhent	***	$2b$12$qDxudEDW5zoRxZyEEqp/ZOLt4RQE6fLfZE3f81w1L24zDGcC2adoK
Omar	omsh	***	$2b$12$V3mfaLckHkJ5mVAYslNvs.KJ3Pb0J8T4tgRQ355y5OolrC3S5IjHq

Considered industry standard

LLMs from within SQL

Large Language Models (LLMs) can be used from within SQL on actual data.

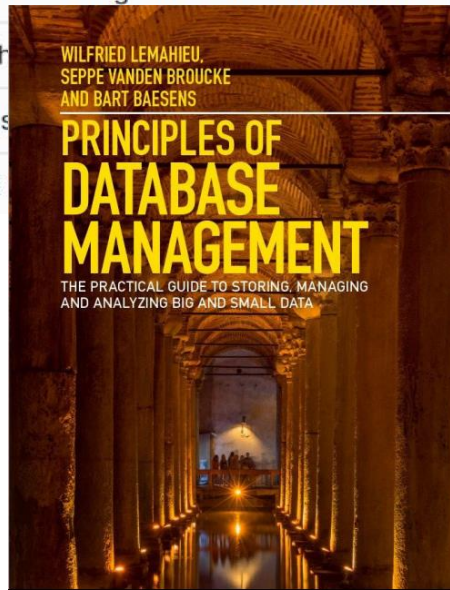
- Large database vendors have added support in the last year
- Enables use of LLMs across large data sets in relational tables

→ SQL keeps getting improved all the time

LLM Example

Book reviews from Amazon

REVIEWER	STARS	REVIEW
DrewD	4	Superb book, but no ANSWERS to Review Questions? Really? A great read on modern database man
Draining Sink	3	Comprehensive but difficult to follow Like other reviewers have said, it does go into a great deal of d
SimonPer	3	cover bends easily very interesting but the cover bends to easily to my liking.
Amazon Customer	2	No solutions/explanations for review questions The content seems to be good but it is unfortunate th
Anne	5	Comprehensive book with high degree of practical orientation I h
ML Enthusiast	5	Comprehensive presentation of data base and data analytics I us
Jan Mendling	5	A Data Science book on Databases This book is a much needed



Source: https://www.amazon.de/Principles-Database-Management-Practical-Analyzing/dp/1107186129

LLM Example

call to LLM

specific model

concat operator

```
-- double-check if the review matches the stars given
select reviewer,
  stars,
  snowflake.cortex.complete('mistral-large', 'On a scale from 1-5, how many stars does
    the following review give according to the sentiment of the text. Output a single
    number only. Output no text. Review: ' || review) as llm_stars,
  review
from amazon_reviews;
```

REVIEWER	STARS	LLM_STARS	REVIEW
DrewD	4	4	Superb book, but no ANSWERS to Review Questions? Really? A great read on mod
Draining Sink	3	3	Comprehensive but difficult to follow Like other reviewers have said, it does go int
SimonPer	3	Based on the sent	cover bends easily very interesting but the cover bends to easily to my liking.
Amazon Customer	2	2	No solutions/explanations for review questions The content seems to be good but
Anne	5	5	Comprehensive book with high degree of practical orientation I have used this boo
ML Enthusiast	5	5	Comprehensive presentation of data base and data analytics I use this book to pre
Jan Mendling	5	5	A Data Science book on Databases This book is a much needed foundational piec

LLM_STARS

```
Based on the sentiment expressed in
the review, it seems the user has
some issues with the product (the
cover bending easily). However, they
also mention that it's "very
interesting", which could suggest
some positive aspects. Given the
mixed sentiment, but with the
negative aspect seemingly
outweighing the positive, I would
rate this review a 3 out of 5.
```

LLM Example

```
-- what to improve per stars given
select stars,
  snowflake.cortex.complete('snowflake-arctic', 'What to improve? Answer in three
    words maximum: ' || listagg(review, '; ')) as improvements
from amazon_reviews
group by stars
order by stars desc;
```

Listagg concatenates all strings of a group

Can use all SQL operators like group by, order by, etc.

# STARS	A IMPROVEMENTS
5	Improve: Practical Examples, Exercises, Open Questions
4	Include answers.
3	Improve clarity, illustrations, and cover durability.
2	Improve accessibility, transparency, and user experience.

Takeaways

SQL Modifications via `insert`, `update`, `delete` statements

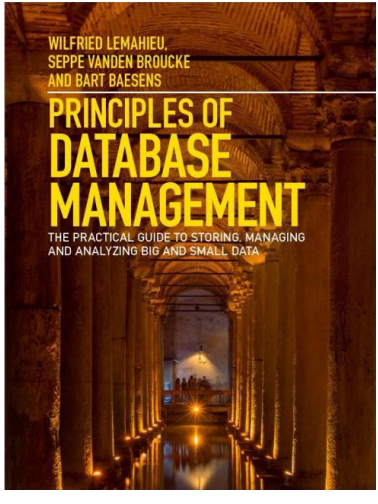
Views and WITH for increasing readability

- Views stored in database, use across queries and sessions, can increase security
- WITH per query only

UDFs extend SQL with more functionality and other programming languages

LLMs are the newest addition

Readings: Chapter 7.3 insert/update/delete, Chapter 7.4



and Window function (SQL) – Wikipedia
and Window Functions – DuckDB